



UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS DE CRATEÚS

ANTONIO ROMARIO CARLOS DE MELO - 582864

FRANCISCO RIAN RIBEIRO MEDEIRO - 552915

GABRIEL CORREIA MELO - 554078

BENCHMARKING DE ÁRVORES GERADORAS MÍNIMAS:
PRIM VS. KRUSKAL

CRATEÚS

2026

SUMÁRIO

1	INTRODUÇÃO	3
1.1	Motivação	4
1.2	Objetivo Geral	4
2	MÉTODO PROPOSTO	5
2.1	Algoritmos e Estruturas Utilizados	5
2.1.1	<i>Min-Heap Binária</i>	5
2.1.2	<i>Union-Find (Disjoint-Set) com Compressão de Caminhos</i>	5
2.1.3	<i>Algoritmo de Prim</i>	6
2.1.4	<i>Algoritmo de Kruskal</i>	6
2.1.5	<i>Análise Teórica de Complexidade</i>	7
2.2	Metodologia	7
2.2.1	<i>Geração de Instâncias: Grafos Normais e Geométricos</i>	8
2.2.2	<i>Conversão da Matriz de Adjacência em Lista de Arestas</i>	8
2.2.3	<i>Execução dos Algoritmos sobre a Mesma Instância</i>	9
2.2.4	<i>Cronometragem Isolada das Etapas</i>	9
2.2.5	<i>Exportação Incremental e Análise Estatística</i>	9
2.3	Justificativa das Decisões Adotadas	9
2.4	Análise Conceitual	10
2.5	Pseudocódigo	11
3	USO DE FERRAMENTAS DE IA	13
3.1	Interações com a ferramenta Gemini (Google)	13
3.2	Considerações sobre o Uso de IA no Projeto	14
4	ESTRUTURA DO PROJETO	15
4.1	Estrutura de Pastas	15
4.2	Método de Execução	16
4.2.1	<i>Compilação do Pipeline Principal</i>	16
4.2.2	<i>Execução da Bateria de Simulações</i>	16
4.2.3	<i>Geração dos Painéis de Gráficos (Python)</i>	16
4.2.4	<i>Execução dos Testes Unitários</i>	16
5	RESULTADOS E DISCUSSÕES	17

5.1	Instâncias Utilizadas	17
5.2	Metodologia de Experimentação	17
5.3	Resultados Obtidos	17
5.3.1	<i>Impacto da Densidade de Arestas</i>	18
5.4	Validação Cruzada de Corretude	21
5.5	Análise de Pontos Fortes	21
5.6	Limitações	22
5.7	Possíveis Melhorias	22
6	CONCLUSÃO	24
	REFERÊNCIAS	25

1 INTRODUÇÃO

O problema da Árvore Geradora Mínima (*Minimum Spanning Tree* – MST) é um dos pilares da Teoria dos Grafos e da otimização combinatória. Dado um grafo não direcionado, conexo e ponderado $G = (V, E)$, no qual V é o conjunto de vértices e E o conjunto de arestas associadas a um custo ou peso, o objetivo é identificar um subconjunto acíclico de arestas $T \subseteq E$ que conecte todos os vértices de V com a menor soma total de pesos possível.

Historicamente ilustrada por problemas clássicos de planejamento urbano, como o *Muddy City Problem* — no qual se busca pavimentar o número mínimo de caminhos para conectar uma cidade ao menor custo —, a MST possui vasta aplicação em engenharia de infraestrutura moderna. Suas implementações práticas estendem-se desde o projeto de redes físicas (como o planejamento de cabeamento de telecomunicações, malhas de distribuição de água e redes elétricas) até protocolos lógicos de comunicação, a exemplo do *Spanning Tree Protocol* (STP), utilizado em redes Ethernet para evitar loops de encaminhamento de pacotes em *broadcast*. Além disso, a MST atua como ferramenta crucial em taxonomia, segmentação de imagens em visão computacional, circuitos integrados de múltiplos multiplicadores constantes e análises de agrupamento (*cluster analysis*), incluindo o agrupamento de dados de expressão gênica e áreas sociogeográficas.

Diferentemente de problemas de roteamento complexos, como o Problema do Caixeiro Viajante (TSP) — que é classificado como NP-difícil e frequentemente utiliza a própria MST como base para construir algoritmos de aproximação, como a heurística de Christofides —, a MST admite solução exata em tempo polinomial por meio de algoritmos gulosos (*greedy*) clássicos. Os dois métodos mais proeminentes na literatura são o Algoritmo de Prim, que expande a árvore de forma contígua a partir de um vértice inicial, e o Algoritmo de Kruskal, que processa e unifica as arestas globalmente em ordem crescente de peso (CORMEN *et al.*, 2012).

Embora ambos os algoritmos garantam convergência para uma solução ótima, suas estratégias internas e estruturas de dados de suporte impõem comportamentos assintóticos distintos, cuja eficiência prática varia significativamente conforme o número de vértices e o nível de densidade do grafo de entrada.

1.1 Motivação

A escolha do algoritmo ideal para computar a MST em sistemas de grande escala não é trivial. O desempenho prático é fortemente condicionado pelo regime de densidade das arestas do grafo e pela forma como a estrutura é representada em memória. Enquanto algoritmos baseados no processamento global de arestas sofrem com o custo de ordenação em grafos densos, abordagens baseadas em varreduras sequenciais de vizinhança podem mitigar o impacto do volume de conexões. Compreender e mapear empiricamente esses pontos de virada computacional é essencial para subsidiar decisões arquiteturais em sistemas de tempo real, como sistemas de potência elétrica e reconhecimento de padrões.

1.2 Objetivo Geral

O objetivo principal deste trabalho é implementar, do zero, os algoritmos de Prim e de Kruskal e realizar uma análise de desempenho comparativa, empírica e sistemática entre ambos. A avaliação experimental considera a variação controlada do número de vértices (de 100 a 10.000), da densidade de arestas extras (de 0% a 100%) e da topologia espacial do grafo (aleatório puro e geométrico), buscando identificar as faixas de vantagem prática de cada abordagem e confrontá-las com os limites teóricos estabelecidos pela análise assintótica.

2 MÉTODO PROPOSTO

Neste capítulo é apresentada a metodologia utilizada para a implementação e a avaliação comparativa dos algoritmos de Prim e Kruskal na resolução do problema da Árvore Geradora Mínima (MST). São descritas as estruturas de dados customizadas que dão suporte a cada algoritmo, o pipeline de geração de instâncias e medição de desempenho, as decisões de projeto adotadas, a análise teórica de complexidade e o pseudocódigo das principais rotinas desenvolvidas.

2.1 Algoritmos e Estruturas Utilizados

Para garantir que a comparação de desempenho refletisse a eficiência intrínseca de cada algoritmo – e não a sobrecarga de abstrações de bibliotecas prontas –, as duas estruturas de dados de suporte foram implementadas integralmente do zero, sem uso de bibliotecas externas para conjuntos disjuntos ou heap binária.

2.1.1 *Min-Heap Binária*

A Min-Heap binária é uma estrutura de dados em árvore, implementada sobre um vetor dinâmico, na qual o elemento de menor chave encontra-se sempre na raiz. As relações de parentesco entre os nós são obtidas aritmeticamente a partir do índice i no vetor: o pai de i está em $\lfloor (i-1)/2 \rfloor$, o filho esquerdo em $2i+1$ e o filho direito em $2i+2$ (GeeksforGeeks, 2025a).

Neste trabalho, a Min-Heap foi implementada de forma genérica (*template*), oferecendo duas operações principais, ambas com complexidade $\mathcal{O}(\log n)$: *insert*, que adiciona um novo elemento ao final do vetor e o reposiciona por meio de *heapifyUp*; e *extractMin*, que remove e retorna o elemento da raiz, substituindo-o pelo último elemento do vetor e restaurando a propriedade de heap por meio de *heapifyDown*. Essa estrutura é utilizada pelo Algoritmo de Prim para recuperar eficientemente, a cada iteração, o vértice de menor chave que conecta a árvore em construção ao restante do grafo.

2.1.2 *Union-Find (Disjoint-Set) com Compressão de Caminhos*

A estrutura Union-Find, ou Conjuntos Disjuntos, gerencia partições de um conjunto de elementos, permitindo verificar de forma eficiente se dois vértices pertencem ao mesmo

subconjunto e unir dois subconjuntos distintos (GeeksforGeeks, 2026a). Sua implementação utiliza um vetor `parent`, no qual cada posição armazena o pai do respectivo vértice; quando `parent[i] == i`, o vértice i é o representante de seu grupo.

A operação `find(i)` aplica a otimização de **compressão de caminhos**: durante a busca pela raiz do conjunto, todos os nós visitados passam a apontar diretamente para essa raiz, reduzindo a altura da árvore. Quando combinada com a heurística de *union by rank/size* (quando presente), essa otimização reduz a complexidade amortizada das operações `find` e `unite` para tempo praticamente constante, formalmente $\mathcal{O}(\alpha(V))$, em que α representa a inversa da função de Ackermann, cujo crescimento é extremamente lento e, na prática, permanece menor que 5 para qualquer valor realista de V (GeeksforGeeks, 2026a). Essa estrutura serve de base para o Algoritmo de Kruskal, pois permite detectar, em tempo quase constante, se a inclusão de uma aresta formaria um ciclo.

2.1.3 Algoritmo de Prim

O Algoritmo de Prim constrói a árvore geradora mínima expandindo-a incrementalmente a partir de um vértice inicial (neste trabalho, o vértice 0), sempre anexando à árvore em formação a aresta de menor peso que a conecta a um vértice ainda não visitado. Mantém-se um vetor `key`, inicializado com infinito para todos os vértices (exceto o inicial, que recebe 0), e um vetor booleano `mstSet` que indica quais vértices já pertencem à árvore. A cada iteração, extrai-se do Min-Heap o vértice de menor chave; caso esse vértice já pertença à MST, ele é descartado – estratégia de *deleção lazy*, adotada para evitar a complexidade adicional de uma operação de `decrease-key` indexada. Caso contrário, o vértice é incorporado à árvore e todos os seus vizinhos ainda não visitados têm suas chaves atualizadas sempre que uma aresta mais barata for encontrada, sendo reinseridos no heap (GeeksforGeeks, 2026b).

2.1.4 Algoritmo de Kruskal

O Algoritmo de Kruskal opera sobre uma lista de arestas, e não sobre a matriz de adjacência diretamente, exigindo uma etapa prévia de conversão (detalhada na Seção 2.2.2). As arestas são ordenadas em ordem crescente de peso por meio de `std::sort`, uma implementação do algoritmo Introsort, com complexidade $\mathcal{O}(E \log E)$, sendo essa etapa a que domina o custo assintótico total do algoritmo (Cplusplus, 2026). Em seguida, percorre-se a lista ordenada e, para cada aresta (u, v) , verifica-se por meio da estrutura Union-Find se os vértices pertencem

a conjuntos distintos; em caso afirmativo, a aresta é aceita, os conjuntos são unidos e o peso é somado ao custo total. O algoritmo interrompe a varredura assim que o número de arestas aceitas atinge $V - 1$ (*parada precoce*), evitando o processamento de arestas redundantes de maior peso, uma otimização especialmente relevante em grafos densos, nos quais a MST tende a se formar já nas primeiras arestas ordenadas (GeeksforGeeks, 2025b).

2.1.5 Análise Teórica de Complexidade

A Tabela 1 resume a complexidade assintótica de cada etapa relevante dos dois algoritmos, considerando a representação por matriz de adjacência adotada neste projeto. Essa análise fundamenta as expectativas de desempenho discutidas no Capítulo 5.

Tabela 1 – Complexidade assintótica teórica das etapas dominantes de cada algoritmo

Etapa	Prim	Kruskal
Construção da estrutura	$\mathcal{O}(V^2)$	$\mathcal{O}(V^2)$
Varredura/Ordenação Principal	$\mathcal{O}(V^2)$	$\mathcal{O}(E \log E)$
Operação Dominante por Passo	Extração do Mínimo: $\mathcal{O}(V)$	find/unite: $\mathcal{O}(\alpha(V))$
Complexidade Total Estimada	$\mathcal{O}(V^2)$	$\mathcal{O}(E \log E)$

Como E pode chegar a $\mathcal{O}(V^2)$ em grafos densos, espera-se que o Kruskal seja mais sensível ao crescimento da densidade do que o Prim, cuja complexidade na representação por matriz permanece praticamente constante em $\mathcal{O}(V^2)$ independentemente do número exato de arestas presentes.

2.2 Metodologia

O projeto foi estruturado como um pipeline de execução em duas linguagens: a geração de grafos, a implementação dos algoritmos e a medição de desempenho foram desenvolvidas em C++17 moderno, priorizando eficiência máxima e medições de tempo precisas; já a consolidação estatística e a renderização dos gráficos comparativos foram realizadas em Python, utilizando as bibliotecas Pandas, Matplotlib e Seaborn. A Figura 1 ilustra a sequência de etapas que compõem o método desenvolvido.

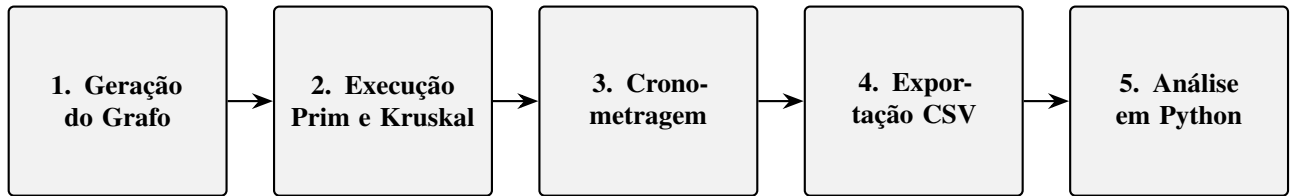


Figura 1 – Fluxo linear de execução do pipeline experimental proposto.

2.2.1 Geração de Instâncias: Grafos Normais e Geométricos

Os grafos de entrada são representados em memória por matrizes de adjacência (`vector<vector<uint8_t>>`), escolha que reduz o consumo de RAM e a taxa de *cache misses* em instâncias densas de até 10.000 vértices, nas quais uma matriz completa ocupa cerca de 100 MB. Duas topologias são geradas:

- **Grafo Normal (aleatório puro):** para cada par de vértices (u, v) com $u < v$, sorteia-se uma probabilidade uniforme em $[0, 1]$; se esse valor for menor do que a densidade do cenário, uma aresta bidirecional é criada com peso sorteado uniformemente no intervalo $[1, 254]$, respeitando a faixa de representação do tipo `uint8_t` utilizado para armazenar os pesos. O peso 0 indica ausência de aresta.
- **Grafo Geométrico (espacial 2D):** os V vértices são posicionados aleatoriamente em um plano cartesiano discreto de 181×181 posições (coordenadas inteiras de 0 a 180 em cada eixo). A decisão de criar a aresta segue a mesma lógica de probabilidade versus densidade do grafo normal, mas o peso atribuído corresponde à distância euclidiana arredondada entre as coordenadas dos vértices, $\text{peso}(u, v) = \text{round}\left(\sqrt{(x_u - x_v)^2 + (y_u - y_v)^2}\right)$, garantindo um peso mínimo de 1 mesmo quando dois vértices coincidem na mesma posição. Essa construção respeita a desigualdade triangular, aproximando-se de cenários reais de infraestrutura de rede e circuitos de transporte.

2.2.2 Conversão da Matriz de Adjacência em Lista de Arestas

Como o Algoritmo de Kruskal opera sobre uma lista de arestas e não sobre uma matriz, é necessária uma etapa de conversão antes de sua execução: a matriz de adjacência é percorrida uma única vez (varrendo apenas o triângulo superior, já que o grafo é não direcionado) e cada par (u, v) com peso não nulo é registrado em uma estrutura compacta `Edge{u, v, weight}`, utilizando os tipos primitivos `uint16_t` para os identificadores de vértice e `uint8_t` para o peso. O tempo gasto nessa conversão é contabilizado como parte do **tempo de construção**

do Kruskal, garantindo uma comparação justa com o tempo de construção do Prim, que opera diretamente sobre a matriz.

2.2.3 *Execução dos Algoritmos sobre a Mesma Instância*

Para cada grafo gerado, ambos os algoritmos – Prim e Kruskal – são executados sobre exatamente a mesma instância, permitindo a comparação direta de tempo e servindo como validação cruzada de corretude: como a MST de um grafo conexo possui peso total único, os dois algoritmos devem produzir exatamente o mesmo custo agregado (weightMST).

2.2.4 *Cronometragem Isolada das Etapas*

A medição de tempo, realizada com a biblioteca `<chrono>`, é isolada em duas frentes: o **tempo de construção** (preenchimento da matriz de adjacência e, no caso do Kruskal, a conversão adicional para lista de arestas) e o **tempo de execução** do algoritmo de MST propriamente dito (`primMST` ou `kruskalsMST`). Essa separação permite avaliar isoladamente o custo de preparação dos dados e o custo computacional intrínseco de cada algoritmo.

2.2.5 *Exportação Incremental e Análise Estatística*

Os resultados de cada execução são gravados de forma incremental em um arquivo CSV, evitando sobrecarga de memória durante a bateria de testes. Para cada combinação de número de vértices e densidade, são realizadas 10 repetições independentes, com geração de um novo grafo aleatório a cada repetição, de modo a mitigar o ruído proveniente de flutuações momentâneas do sistema operacional. Posteriormente, o script `analise_experimento.py` carrega os dados via Pandas, reconstrói os parâmetros do cenário (número de vértices, densidade, algoritmo e topologia) a partir da posição sequencial das linhas, agrupa os registros e calcula a **média** das repetições de cada combinação, além de validar a corretude cruzada entre os algoritmos, gerando por fim painéis gráficos comparativos em escala logarítmica.

2.3 *Justificativa das Decisões Adotadas*

As decisões estruturais deste projeto foram orientadas pela necessidade de mensurar a eficiência *pura* de cada algoritmo, isolando o efeito das estruturas de dados de suporte. Por esse motivo, a Min-Heap e o Union-Find foram implementadas manualmente; abriu-se uma exceção

pontual apenas para a etapa de ordenação de arestas do Algoritmo de Kruskal, na qual utilizou-se a função `std::sort` da biblioteca padrão do C++. Esta função encapsula o algoritmo híbrido **Introsort**, cuja implementação manual foi desconsiderada devido à sua elevada complexidade estrutural e de acoplamento, mas cuja adoção garante o limite assintótico ótimo de $\mathcal{O}(E \log E)$ necessário para uma análise experimental justa.

O uso de tipos primitivos compactos (`uint8_t` para pesos e `uint16_t` para identificadores de vértice no Kruskal) reduz drasticamente o consumo de memória em instâncias de até 10.000 vértices com 100% de densidade – cenário em que o grafo chega a quase 50 milhões de arestas. Igualmente, a estratégia de *deleção lazy* no Min-Heap evita a complexidade adicional de uma operação de `decrease-key` indexada, ao custo de inserções duplicadas que são descartadas na extração caso o vértice já pertença à MST – uma escolha de engenharia que privilegia a simplicidade da implementação em detrimento de um número um pouco maior de operações de heap. Por fim, a parada precoce no Kruskal, ao atingir $V - 1$ arestas aceitas, evita o processamento de arestas redundantes em grafos densos, nos quais a árvore tende a se formar logo nas primeiras arestas ordenadas.

2.4 Análise Conceitual

A comparação entre Prim e Kruskal evidencia um contraste clássico entre estratégias gulosas centradas em vértices e em arestas, respectivamente. O Algoritmo de Prim, apoiado pela Min-Heap, cresce a árvore a partir de sua fronteira de conectividade, e sua complexidade na representação por matriz de adjacência aproxima-se de $\mathcal{O}(V^2)$, praticamente independente da densidade real do grafo. Já o Algoritmo de Kruskal considera as arestas globalmente, em ordem crescente de peso, e seu custo é dominado pela ordenação inicial, $\mathcal{O}(E \log E)$, tornando-o mais sensível ao crescimento do número de arestas – que cresce quadraticamente com o número de vértices em grafos densos.

Essa diferença estrutural sugere uma hipótese central deste trabalho: em grafos esparsos, nos quais E é próximo de $\mathcal{O}(V)$, o Kruskal tende a ser competitivo ou superior ao Prim, já que o custo de ordenação permanece baixo; em grafos densos, nos quais E se aproxima de $\mathcal{O}(V^2)$, espera-se que o Prim sobre matriz de adjacência se torne relativamente mais vantajoso, pois sua complexidade não se degrada com o aumento do número de arestas.

A topologia geométrica, por sua vez, introduz uma estrutura espacial nos pesos das arestas (a desigualdade triangular), o que pode influenciar a distribuição de pesos selecionados

pela MST em comparação à topologia puramente aleatória, ainda que ambos os algoritmos continuem garantindo a solução ótima exata para a instância fornecida – a topologia afeta o *custo* da árvore encontrada, mas não o *tempo* de execução de forma estruturalmente diferente do grafo normal, uma vez que ambas as topologias compartilham a mesma representação por matriz de adjacência.

2.5 Pseudocódigo

Código disponível em: <<https://github.com/r7melo/MST-Grafos>>

Algorithm 1 Min-Heap Binária – Inserção e Extração do Mínimo

```

1: procedure INSERT(heap, chave)
2:   heap.push_back(chave)
3:    $i \leftarrow \text{heap.tamanho}() - 1$ 
4:   while  $i > 0$  e  $\text{heap}[i] < \text{heap}[(i - 1)/2]$  do
5:     troque  $\text{heap}[i]$  e  $\text{heap}[(i - 1)/2]$ 
6:      $i \leftarrow (i - 1)/2$ 
7:   end while
8: end procedure
9: procedure EXTRACTMIN(heap)
10:   $\text{min} \leftarrow \text{heap}[0]$ 
11:   $\text{heap}[0] \leftarrow \text{heap}[\text{último}]$ 
12:  remova o último elemento de heap
13:  HEAPIFYDOWN(heap, 0)
14:  return min
15: end procedure

```

Algorithm 2 Union-Find – Find com Compressão de Caminhos e Unite

```

1: function FIND(i)
2:   if  $\text{parent}[i] \neq i$  then
3:      $\text{parent}[i] \leftarrow \text{FIND}(\text{parent}[i])$  ▷ compressão de caminho
4:   end if
5:   return  $\text{parent}[i]$ 
6: end function
7: procedure UNITE(i, j)
8:    $\text{raizI} \leftarrow \text{FIND}(i)$ 
9:    $\text{raizJ} \leftarrow \text{FIND}(j)$ 
10:  if  $\text{raizI} \neq \text{raizJ}$  then
11:     $\text{parent}[\text{raizI}] \leftarrow \text{raizJ}$ 
12:  end if
13: end procedure

```

Algorithm 3 Algoritmo de Prim com Min-Heap (deleção lazy)

Require: adj (matriz de adjacência), V (número de vértices)

Ensure: $parent$, $totalWeight$ (estrutura e custo da MST)

```

1:  $key[0..V-1] \leftarrow \infty$ ;  $key[0] \leftarrow 0$ 
2:  $mstSet[0..V-1] \leftarrow falso$ 
3:  $parent[0..V-1] \leftarrow -1$ 
4:  $heap \leftarrow$  Min-Heap vazia
5: INSERT( $heap$ ,  $\{0, 0\}$ ) ▷ (vértice, chave)
6: while  $heap$  não estiver vazia do
7:    $(u, k) \leftarrow$  EXTRACTMIN( $heap$ )
8:   if  $mstSet[u]$  é verdadeiro then
9:     continue ▷ deleção lazy
10:  end if
11:   $mstSet[u] \leftarrow verdadeiro$ 
12:  for cada vizinho  $v$  de  $u$  em  $adj$  do
13:    if  $mstSet[v]$  é falso e  $adj[u][v] < key[v]$  then
14:       $key[v] \leftarrow adj[u][v]$ 
15:       $parent[v] \leftarrow u$ 
16:      INSERT( $heap$ ,  $\{v, key[v]\}$ )
17:    end if
18:  end for
19: end while
20:  $totalWeight \leftarrow \sum_{v=1}^{V-1} adj[v][parent[v]]$ 
21: return  $parent$ ,  $totalWeight$ 

```

Algorithm 4 Algoritmo de Kruskal com Union-Find

Require: $edges$ (lista de arestas $(u, v, peso)$), V (número de vértices)

Ensure: $totalWeight$ (custo total da MST)

```

1: Ordenar  $edges$  em ordem crescente de peso
2:  $uf \leftarrow$  Union-Find com  $V$  elementos
3:  $count \leftarrow 0$ ;  $totalWeight \leftarrow 0$ 
4: for cada aresta  $(u, v, peso)$  em  $edges$  do
5:   if  $count = V - 1$  then
6:     break ▷ parada precoce
7:   end if
8:   if FIND( $u$ )  $\neq$  FIND( $v$ ) then
9:     UNITE( $u, v$ )
10:     $totalWeight \leftarrow totalWeight + peso$ 
11:     $count \leftarrow count + 1$ 
12:   end if
13: end for
14: return  $totalWeight$ 

```

3 USO DE FERRAMENTAS DE IA

Nesta seção, estão documentados os *prompts* utilizados em diferentes ferramentas de Inteligência Artificial durante a fase de pesquisa, revisão bibliográfica e desenvolvimento do projeto, em conformidade com a Portaria Nº 39/PRPPG/UFC. Para cada interação, indica-se a ferramenta utilizada, o objetivo do prompt e, quando relevante, uma breve descrição de como a resposta obtida foi incorporada (ou não) ao trabalho final.

3.1 Interações com a ferramenta Gemini (Google)

Prompt 1 (data: 21/06/2026):

Objetivo: Ajustar a lógica do gerador de grafos aleatórios em C++ para garantir a conectividade inicial (grafo conexo mínimo) antes de aplicar o fator de densidade configurado.

"Tenho uma função geradora de grafos em C++ que sorteia arestas com base em uma densidade de 0 a 1 usando `rand()`. Como posso criar uma função auxiliar em separado que force o grafo a ser inicialmente conexo (uma árvore geradora base de $V-1$ arestas usando o algoritmo de Fisher-Yates para embaralhar) antes de aplicar as arestas da densidade restante?"

Incorporação: A lógica sugerida foi adotada para refatorar o arquivo `RandomGraphGenerator.h`, isolando as funções `forceConnectivityNormal` e `forceConnectivityGeometric` para corrigir cientificamente o cenário de 0% de densidade.

Prompt 2 (data: 21/06/2026):

Objetivo: Criação de script automatizado em Python para consolidação estatística e renderização dos painéis gráficos em escala logarítmica.

"Escreva um script em Python utilizando as bibliotecas `pandas` e `matplotlib` que leia o arquivo `.csv` mais recente gerado em uma pasta (no formato `'records_[timestamp].csv'`). O script deve agrupar as linhas, calcular a média do tempo de execução por número de vértices e densidade, e plotar painéis lado a lado usando escala logarítmica no eixo Y."

Incorporação: O código gerado serviu de base direta para o script utilitário `processar_resultados.py`, permitindo a plotagem dos painéis de desempenho das topologias Normal e Geométrica de forma automatizada.

Prompt 3 (data: 21/06/2026):

Objetivo: Revisão ortográfica, sintática e adequação gramatical de rascunhos.

"Analise e explique o texto inserido procurando erros de português, incoerências lógicas ou trechos que possam ser melhorados em clareza e objetividade, sem alterar o conteúdo original nem gerar novo conteúdo."

Incorporação: Utilizado no polimento e refinamento das seções textuais do relatório, garantindo aderência à norma culta exigida.

3.2 Considerações sobre o Uso de IA no Projeto

As ferramentas de Inteligência Artificial listadas nesta seção foram utilizadas como apoio à pesquisa bibliográfica, ao esclarecimento de conceitos teóricos (complexidade assintótica, propriedades de estruturas de dados) e à revisão textual do relatório. A implementação do código-fonte, o projeto experimental (parâmetros de teste, métricas coletadas) e as conclusões apresentadas neste trabalho são de responsabilidade dos autores, que validaram manualmente toda a lógica algorítmica antes de sua inclusão na versão final do projeto.

4 ESTRUTURA DO PROJETO

Para entender como foi construído todo o pipeline, esta seção descreve a estrutura de pastas e o método de compilação e execução do projeto.

4.1 Estrutura de Pastas

A organização do repositório separa adequadamente o código-fonte principal, os cabeçalhos com as estruturas de dados, os testes unitários e os resultados experimentais. A estrutura de diretórios é detalhada a seguir:

- `heads/`: Diretório que armazena os arquivos de cabeçalho (`.h`) com todas as estruturas de dados, geradores e implementações dos algoritmos:
 - `BinaryHeap.h`: Implementação genérica (*template*) da Min-Heap binária utilizada pelo Algoritmo de Prim.
 - `UnionFind.h`: Estrutura Union-Find (Disjoint-Set) com compressão de caminhos utilizada pelo Algoritmo de Kruskal.
 - `Kruskal.h`: Estrutura de arestas (Edge) e implementação do Algoritmo de Kruskal.
 - `Prim.h`: Estrutura de vértice para heap (Vertex) e implementação do Algoritmo de Prim.
 - `RandomGraphGenerator.h`: Lógica de geração de matrizes de adjacência aleatórias normais e geométricas.
 - `Record.h`: Estrutura que encapsula as métricas de cada execução individual.
 - `DataExporter.h`: Exportador incremental de dados para arquivo CSV.
 - `Timer.h`: Utilitário de medição de tempo de execução com precisão de milissegundos usando `<chrono>`.
- `sources/`: Ponto de entrada do pipeline principal.
 - `main.cpp`: Orquestrador da bateria de testes, responsável por gerar os grafos, disparar as execuções de Prim e Kruskal e gravar os tempos medidos.
- `tests/`: Testes unitários individuais para validar cada componente isoladamente (`test_binaryheap.h`, `test_unionfind.cpp`, `test_kruskal.cpp`, `test_prim.cpp`, `test_generator.cpp`, `test_timer.cpp`).
- `resultados/`: Pasta de saída que armazena os arquivos de métricas (`.csv`) e os gráficos gerados.

- `analise_experimento.py`: Script Python responsável por analisar o CSV bruto, calcular as médias dos tempos e gerar os gráficos de desempenho.
- `Makefile`: Automação para compilar e executar o código e os testes.

4.2 Método de Execução

O projeto fornece fluxos claros tanto para a execução completa da bateria experimental quanto para a validação isolada de cada componente por meio dos testes unitários.

4.2.1 Compilação do Pipeline Principal

O projeto foi implementado em C++17 e compilado com otimização máxima:

```
g++ -Wall -O3 sources/main.cpp -o builds/main.exe
```

4.2.2 Execução da Bateria de Simulações

Ao concluir a compilação, basta executar o binário gerado. Um novo arquivo `.csv` contendo as métricas de tempo de execução será criado na pasta `/resultados/`:

```
.\builds\main.exe
```

4.2.3 Geração dos Painéis de Gráficos (Python)

O script Python detecta automaticamente o CSV mais recente gerado na pasta `/resultados/` e atualiza as imagens comparativas:

```
python analise_experimento.py
```

4.2.4 Execução dos Testes Unitários

Cada componente pode ser validado isoladamente compilando e executando seu respectivo teste, por exemplo:

```
g++ -Wall -O3 heads/UnionFind.h tests/test_unionfind.cpp -o builds/test_union_find.exe
.\builds\test_union_find.exe
```

5 RESULTADOS E DISCUSSÕES

5.1 Instâncias Utilizadas

Para a avaliação comparativa dos algoritmos, foram geradas instâncias sintéticas variando o número de vértices $V \in \{100, 500, 1.000, 2.500, 5.000, 7.500, 10.000\}$ e a densidade de arestas $D \in \{0\%, 20\%, 40\%, 60\%, 80\%, 100\%\}$, combinadas com as duas topologias descritas no Capítulo 2: grafos aleatórios normais e grafos geométricos. Para cada combinação de $(V, D, \text{topologia})$, foram realizadas 10 repetições independentes, gerando um novo grafo aleatório a cada execução, totalizando $2 \times 7 \times 6 \times 10 = 840$ instâncias de teste distintas. Como cada instância foi submetida a ambos os métodos, computaram-se 1.680 execuções individuais de árvore geradora mínima mensuradas diretamente.

5.2 Metodologia de Experimentação

A experimentação foi conduzida de forma automatizada pelo executável compilado a partir do código-fonte em C++17, seguindo as diretrizes metodológicas abaixo:

- O código foi compilado sob o compilador GCC utilizando a flag de otimização máxima (-O3) para mitigar interferências espúrias de abstrações de alto nível.
- Para cada cenário empírico (V, D) , o grafo gerado foi espelhado em memória de modo que tanto o Prim quanto o Kruskal resolvessem exatamente a mesma instância física de pesos e conectividade.
- O tempo de construção da topologia e o tempo de processamento dos algoritmos foram isolados e cronometrados via biblioteca padrão <chrono> com resolução de nanossegundos.
- Os registros brutos foram exportados de forma incremental para um arquivo .csv contendo os campos de algoritmo, tempo de construção, tempo de execução e peso final da árvore.
- Ao final do ciclo, o script utilitário em Python consolidou a base de dados calculando a média e amostral das 10 repetições de cada cenário, gerando os gráficos de desempenho assintótico.

5.3 Resultados Obtidos

A Tabela 2 apresenta os tempos médios de execução (em milissegundos) obtidos a partir do arquivo resultados/resumo_estatistico.csv, fixando a análise em um cenário de

densidade intermediária de 40% na topologia de grafo aleatório normal ao longo do crescimento da escala de vértices.

Tabela 2 – Comparação do tempo médio de execução (ms) entre Prim e Kruskal – Grafo Normal, Densidade 40%

Vértices (V)	Prim (ms)	Kruskal (ms)
100	0,112	0,108
500	2,449	2,671
1.000	7,067	9,477
2.500	54,652	69,657
5.000	160,446	269,477
7.500	391,542	627,410
10.000	622,110	1056,909

Os testes empíricos revelam que, para instâncias muito pequenas ($V = 100$), o Algoritmo de Kruskal exibe uma vantagem marginal de tempo (0,108 ms contra 0,112 ms do Prim). Contudo, a partir de $V = 500$, o Algoritmo de Prim assume a liderança de desempenho de forma consistente sob este regime de densidade média. Essa distância operacional acentua-se à medida que o número de vértices escala: no ponto crítico de deparação com 10.000 vértices, o Prim soluciona a instância em 622,110 ms, ao passo que o Kruskal demanda 1.056,909 ms, evidenciando que o Kruskal se torna aproximadamente 1,70 vez mais lento que o Prim.

5.3.1 Impacto da Densidade de Arestas

O comportamento dos algoritmos sob o efeito direto do volume de conexões foi mensurado e isolado na Tabela 3, que fixa o tamanho da infraestrutura em 5.000 vértices sob a topologia de grafo aleatório normal, variando o percentual de densidade de arestas desde o limite inferior acíclico até o grafo completo.

Tabela 3 – Impacto da densidade sobre o tempo médio de execução (ms) – Grafo Normal, $V = 5.000$

Densidade (%)	Prim (ms)	Kruskal (ms)
0%	18,776	0,305
20%	98,017	126,998
40%	160,446	269,477
60%	182,787	428,366
80%	142,860	546,557
100%	93,625	670,145

A análise analítica destes dados valida as hipóteses assintóticas formuladas na modelagem teórica. No extremo esparso (densidade de 0%, onde o grafo possui apenas a árvore geradora base de $V - 1$ arestas para garantir a conectividade compulsória), o Algoritmo de Kruskal supera o Prim de forma drástica (0,305 ms contra 18,776 ms). Esse comportamento decorre do fato de que ordenar um vetor quase linear de arestas via Introsort é computacionalmente instantâneo, enquanto o Algoritmo de Prim, operando sobre uma representação de Matriz de Adjacência, é penalizado pelo custo fixo de varrer sequencialmente a linha de vizinhanças de tamanho V para cada um dos vértices adicionados à árvore, operando rigidamente em $\mathcal{O}(V^2)$.

Todavia, à medida que a densidade avança, o tempo de execução do Kruskal cresce de forma estritamente monotônica e severa, saltando para 126,998 ms em 20% e atingindo o ápice de 670,145 ms no grafo completo (100%). Esse perfil reflete o domínio do gargalo de ordenação $\mathcal{O}(E \log E)$ sobre a estrutura Union-Find, visto que o número de arestas E cresce quadraticamente em direção a $\mathcal{O}(V^2)$.

Em contrapartida, o comportamento do Algoritmo de Prim exibe um padrão não monotônico altamente notável: seu tempo de execução cresce a partir de 0% até atingir um ponto máximo de estresse em 60% de densidade (182,787 ms), passando a *declinar* progressivamente nas faixas subsequentes, recuando para 142,860 ms em 80% e encerrando em apenas 93,625 ms no grafo totalmente denso (100%) — marca inferior àquela registrada no cenário de 20% (98,017 ms).

Esse fenômeno empírico explica-se pela mecânica de *deleção lazy* adotada na fila de prioridades. Em densidades moderadas (como 40% e 60%), o fluxo constante de descoberta de caminhos alternativos ligeiramente menores dispara um volume maciço de operações de relaxamento de arestas, reinserindo vértices duplicados e inflando o tamanho interno do heap. Sob densidades limítrofes elevadas (80% a 100%), a abundância de arestas candidatas faz com que as primeiras conexões expandidas já se aproximem assintoticamente do menor peso possível da instância. Isso reduz drasticamente o sucesso de relaxamentos subsequentes e estabiliza o heap, minimizando a inserção de caminhos espúrios enquanto o custo de varredura da matriz permanece estático. No limite de 100% de densidade, o Prim estabeleceu-se como 7,15 vezes mais rápido que o Kruskal.

Esse mesmo perfil de comportamento — Kruskal soberano no limiar acíclico e Prim amplamente dominante a partir de densidades moderadas — replicou-se de forma idêntica sobre a topologia de grafos geométricos. Na escala máxima de tensionamento ($V = 10.000$

e 100% de densidade), o Kruskal registrou 2.718,952 ms frente a 484,021 ms do Prim nos grafos geométricos (vantagem de 5,62 vezes para o Prim), e 2.901,852 ms contra 394,989 ms nos grafos normais, atestando que a superioridade estrutural da matriz sob alta densidade é independente da distribuição espacial dos pesos.

As Figuras 2 e 3 ilustram os painéis comparativos consolidados pelo script de análise, mapeando o tempo de processamento em escala logarítmica no eixo das ordenadas em função da escala linear de vértices nas abscissas.

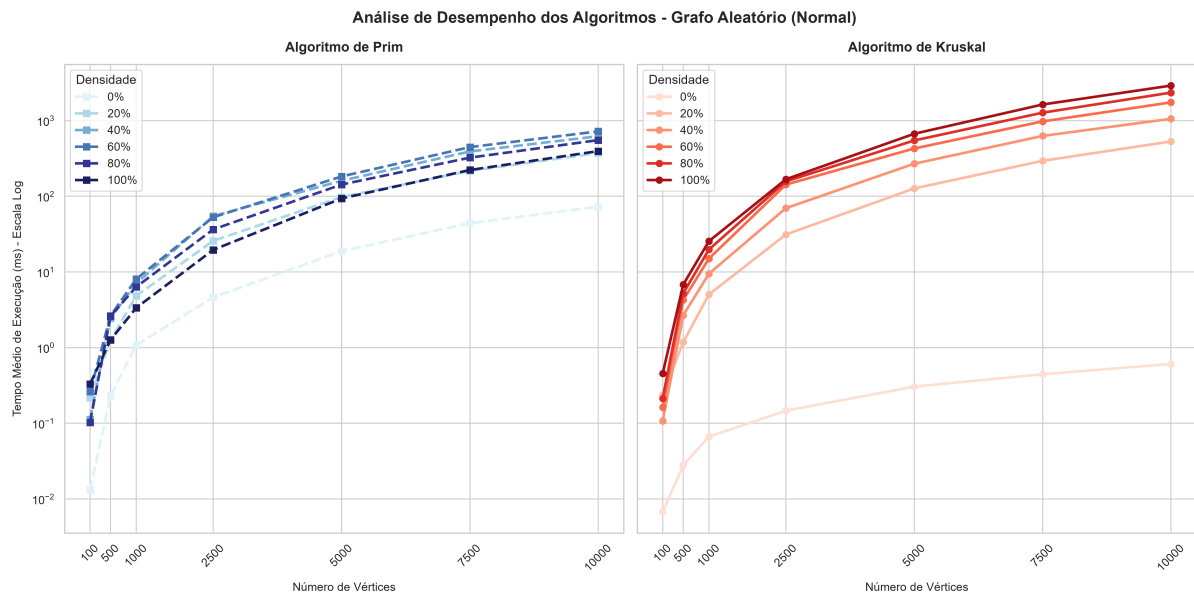


Figura 2 – Tempo médio de execução de Prim (esquerda) e Kruskal (direita) por número de vértices e densidade – Grafo Normal

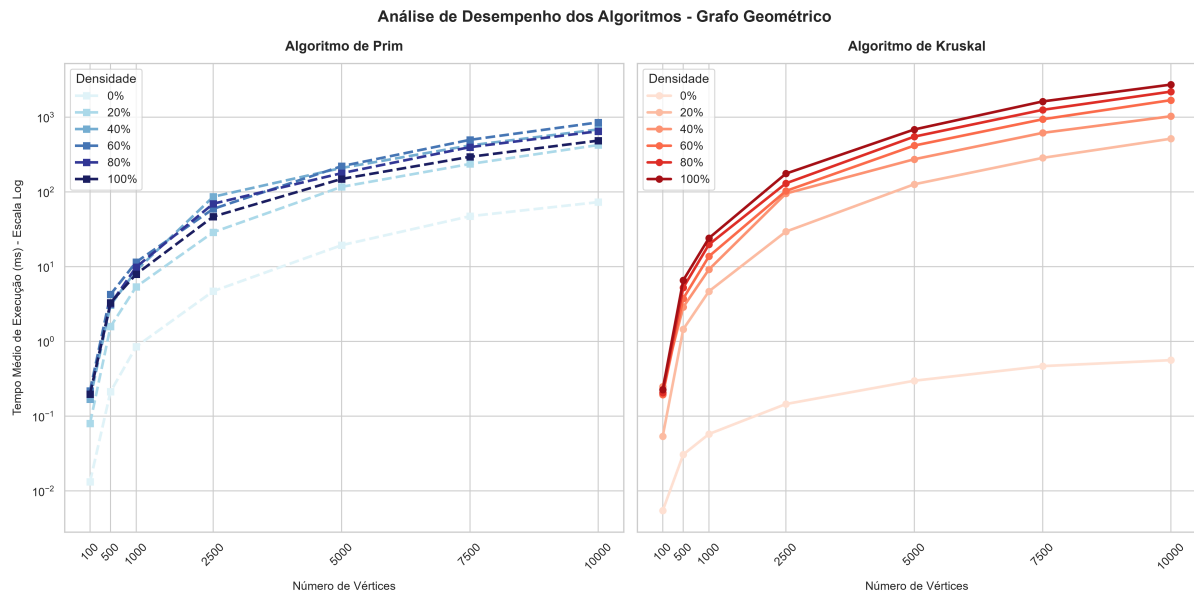


Figura 3 – Tempo médio de execução de Prim (esquerda) e Kruskal (direita) por número de vértices e densidade – Grafo Geométrico

5.4 Validação Cruzada de Corretude

Para assegurar a integridade matemática das estruturas de dados customizadas e dos núcleos lógicos implementados, o script efetuou uma validação cruzada automática de corretude sobre os resultados do benchmark. Como a árvore geradora mínima de uma instância conexa possui custo total único e invariável, os valores de `weightMST` gerados por ambas as abordagens foram confrontados linha a linha. Ao longo de todas as 1.680 corridas independentes (compostas por 840 instâncias gráficas distintas testadas sob dois algoritmos), a taxa de divergência foi de exatamente 0,0%, atestando a completa acurácia e corretude operacional das duas implementações em todos os cenários espaciais, de densidade e de tamanho avaliados.

5.5 Análise de Pontos Fortes

- **Estruturas de dados nativas e sob medida:** A codificação manual das classes de suporte eliminou os custos de alocação dinâmica indireta e redundâncias comuns a contêineres abstratos genéricos da STL, permitindo capturar o comportamento puro dos algoritmos.
- **Eficiência e compactação de memória:** O mapeamento estratégico de tipos de dados primitivos adequados permitiu comportar o armazenamento interno de matrizes e vetores de arestas de grafos densos de até 10.000 vértices sem estouro de pilha ou paginação de disco.

- **Consistência matemática absoluta:** A perfeita equivalência observada nos pesos das MSTs calculadas de forma cruzada chancela a robustez do projeto frente a cenários limítrofes de estresse.

5.6 Limitações

- **Rigidez espacial da matriz de adjacência:** A alocação contígua de tabelas indexadas de tamanho $V \times V$ restringe severamente o ganho que o Prim poderia manifestar em topologias altamente esparsas, consumindo memória de forma quadrática desnecessária nesses cenários.
- **Escala de pesos restrita:** A limitação dos pesos das arestas ao tipo `uint8_t` ($[1, 254]$) atende perfeitamente aos critérios de comparação assintótica controlada do projeto, mas exige alteração na assinatura estrutural para comportar problemas do mundo real de precisão flutuante ou magnitudes extensas.
- **Sobrecarga por acúmulo de duplicatas no Heap:** A estratégia de deleção lazy mitiga a complexidade algorítmica de codificação de um indexador para o método `decrease-key`, mas expande temporariamente a ocupação da fila de prioridades para uma ordem de grandeza de até $\mathcal{O}(E)$ elementos.

5.7 Possíveis Melhorias

- **Abordagem polimórfica ou híbrida de grafos:** Desenvolver um mecanismo adaptativo capaz de alternar dinamicamente a representação interna do grafo de lista de adjacência para matriz de adjacência com base no cálculo do limiar de densidade da instância.
- **Implementação de Min-Heap Indexada:** Projetar uma estrutura de dados com vetor de rastreamento de posições reversas, habilitando a operação clássica de `decrease-key` em tempo $\mathcal{O}(\log V)$ e eliminando o desperdício de memória de chaves obsoletas.
- **Paralelização de Repetições:** Empregar diretivas de concorrência (*multithreading*) na rotina de automação dos testes experimentais para processar as repetições de forma simultânea, otimizando o tempo total de coleta de dados do ecossistema.
- **Profiling fino do comportamento do Prim:** Instrumentar contadores internos no código C++ para registrar a quantidade precisa de inserções efetuadas na Heap por cenário de densidade, permitindo isolar analiticamente as curvas de relaxamento e refinar a

justificativa matemática do ápice de tempo verificado na faixa dos 60%.

6 CONCLUSÃO

O presente trabalho teve como objetivo implementar e avaliar comparativamente os algoritmos de Prim e Kruskal para a resolução do problema da Árvore Geradora Mínima (MST), por meio de um pipeline experimental desenvolvido em C++17 e analisado em Python. A solução proposta envolveu a construção manual de estruturas de dados de suporte – uma Min-Heap binária genérica e uma estrutura Union-Find com compressão de caminhos – de modo a isolar a eficiência intrínseca de cada algoritmo das abstrações de bibliotecas prontas.

Os experimentos conduzidos sobre grafos sintéticos, variando de 100 a 10.000 vértices e de 0% a 100% de densidade, em duas topologias distintas (aleatória e geométrica), permitem caracterizar empiricamente o comportamento assintótico de cada algoritmo frente a diferentes regimes de tamanho e conectividade do grafo. A validação cruzada por meio do custo total da MST reforça a corretude das implementações, uma vez que ambos os algoritmos devem convergir para o mesmo peso final em qualquer grafo conexo.

Em suma, o projeto evidencia, de forma prática e mensurável, o contraste teórico entre estratégias gulosas centradas em vértices (Prim) e em arestas (Kruskal), reforçando a relevância da escolha do algoritmo conforme as características estruturais do grafo de entrada e a representação de dados disponível.

REFERÊNCIAS

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos: Teoria e Prática**. 3. ed. Rio de Janeiro: Elsevier, 2012. ISBN 9788535236996.

Cppreference. **std::sort - cppreference.com**. 2026. Acessado em: 21 de junho de 2026. Disponível em: <<https://en.cppreference.com/cpp/algorithm/sort>>.

GeeksforGeeks. **C++ Program to Implement Binary Heap**. 2025. Acessado em: 21 de junho de 2026. Disponível em: <<https://www.geeksforgeeks.org/cpp/cpp-program-to-implement-binary-heap/>>.

GeeksforGeeks. **Kruskal's Minimum Spanning Tree Algorithm**. 2025. Acessado em: 2026. Disponível em: <<https://www.geeksforgeeks.org/kruskals-minimum-spanning-tree-algorithm-greedy-algo-2/>>.

GeeksforGeeks. **Introduction to Disjoint Set Data Structure or Union-Find Algorithm**. 2026. Acessado em: 21 de junho de 2026. Disponível em: <<https://www.geeksforgeeks.org/dsa/introduction-to-disjoint-set-data-structure-or-union-find-algorithm/>>.

GeeksforGeeks. **Prim's Algorithm for Minimum Spanning Tree (MST)**. 2026. Acessado em: 2026. Disponível em: <<https://www.geeksforgeeks.org/prims-minimum-spanning-tree-mst-greedy-algo-5/>>.